Computer Vision - Exam Preparation Q&A

Computer Vision Students

April 3, 2026

Contents

Lecture 1-3: Introduction and Image Processing Basics **5**

In a CV systems pipeline, how do low-level tasks compare versus mid and high level tasks? 5

Lecture 4-5: Single pixel ops, img histogram, spatial filtering **5**

Define point operations and explain the difference between Logarithmic and Power-Law (Gamma) transformations 5

Compare and contrast "Contrast Stretching" and "Thresholding". 5

What is the main limitation of Histogram Equalization, and why might we use Histogram Specification (Matching) instead? 6

Explain the concepts of histogram and its equalization. A) Define an image histogram. B) What is the goal of histogram equalization and why is it useful? C) Outline the procedure for deriving the equalization function, providing an example. 6

Describe the mathematical procedure for performing Histogram Specification. . 7

Lecture 6: Non-linear filters and frequency domain **7**

Explain the principle of the median filter and its primary advantage over linear smoothing filters. 7

How does the Bilateral Filter achieve edge-preserving smoothing, and how does it differ from a standard Gaussian filter? 8

In the context of 2D FT, how do low and high frequency translate to the visual components of an image? 8

Compare ideal LPF and Butterworth LPF (BLPF) in the frequency domain. Why is BLPF generally preferred in practice? 8

How are geometric spatial transformations mathematically represented and applied to an image? 9

Lecture 7: Filtering in frequency and geometrical transformations **9**

Analyze the two steps required by a geometric trasform. 9

How is a point represented in standard 2D coordinates vs homogeneous coordinates? And what is the structural advantage of using homogeneous coordinates vs standard cartesian ones when performing geometric translations? 9

What makes an affine transform different from a basic linear transform, and what specific properties does it preserve? 10

When analyzing an image for edges, what is the conceptual difference between using a derivative filter versus a true edge detection algorithm? 10

Lecture 8: Detecting edges and lines **10**

How do first-order and second-order derivatives differ in their approach to edge detection? 10

In the Canny algorithm pipeline, what distinct structural problems do Non-Maxima Suppression and Hysteresis Thresholding solve? 10

How does the choice of the Gaussian kernel size (σ) in Step 1 dictate the final output of the Canny edge detector? 11

Why is Canny considered a complete "edge detector algorithm" rather than just a simple derivative filter? 11

When building the parameter space for the Hough Transform, why is the normal representation ($x \cos\theta + y \sin\theta = \rho$) preferred over the standard Cartesian equation ($y = ax + b$)? 11

In the Hough Transform algorithm, the parameter space (ρ, θ) must be quantized into accumulation cells. What is the engineering trade-off between using few cells versus many cells? 12

How does the Generalized Hough Transform differ conceptually from the standard version, and what is its main computational drawback? 12

Why do we prefer the Hough Transform over a naive approach to find lines? (Computational Complexity) 12

Lecture 9: Morphological Operations and Thresholding 13

In morphological operations, how does Erosion differ from Dilation in terms of their physical effect on foreground shapes and component connectivity? . . 13

Structurally and mathematically, what is the exact difference between an Opening operation versus a Closing operation? 13

When dealing with imperfect binary images, how does Opening perform versus Closing in terms of noise removal and shape repair? 13

Since morphological operations are not cumulative, when is it appropriate to use [Opening then Closing] versus [Closing then Opening]? 14

In the context of morphological descriptions, what is the conceptual difference between the image set versus the structuring element? 14

When segmenting an image, why might variable (local) thresholding be preferred over global thresholding? 14

How does Otsu's method use inter-class variance versus intra-class variance to find the optimal threshold? 15

In the context of Otsu's algorithm, what is the conceptual difference between the cumulative mean $m(k)$ and the mean intensity value $m_1(k)$ for Class 1? 15

How does image noise and object size affect the standard Otsu's method versus an approach that incorporates image smoothing? 15

Lecture 10: Segmentation by clustering 16

Compare region growing with thresholding-based segmentation, and outline the core steps of a basic region growing algorithm. 16

Explain the core principle of Watershed segmentation. 16

What is the primary failure mode of Watershed segmentation, and what is the standard algorithmic enhancement used to resolve it? 17

Define the objective function of the K-means clustering algorithm and outline the standard iterative optimization approach (Lloyd's algorithm). 17

Critically evaluate the structural assumptions of K-means segmentation, and analyze the trade-offs of including spatial coordinates in the feature vector versus using color/intensity alone. 18

Explain the conceptual foundation of density-based clustering using Kernel Density Estimation and outline how it theoretically addresses K-means' limitations. 19

Formally define the mathematical construction of a Probability Density Function using Kernel Density Estimation, and explain the primary computational drawback of this method. 19

Lecture 1-3: Introduction and Image Processing Basics

In a CV systems pipeline, how do low-level tasks compare versus mid and high level tasks?

Low-level processing focuses on image filtering, gray-level/color processing, and image enhancement (e.g., single-pixel transforms, DFT, local operators). Examples include single pixel and histogram-based transform, linear filters, non-linear filters.

Mid-level processing is based on extracting more complex models from the image, such as line detection, blob detection, salient features/keypoints, and edge detection algorithms. Examples include derivate, corner detection, hough transform, segmentation, feature detection (Keypoints detection and feature descriptor calculation, feature matching).

High-level processing includes object detection and recognition, image (semantic) segmentation, ML for CV, projective geometry, image formation and cameras, camera calibration, DL for CV.

Lecture 4-5: Single pixel ops, img histogram, spatial filtering

Define point operations and explain the difference between Logarithmic and Power-Law (Gamma) transformations

Point ops are spatial domain image processing techniques where the new value of a pixel depends strictly on the intensity value of that same pixel in the original image, without considering its neighbors. This is expressed as s = T(r), where r is the original intensity, s is the new intensity, and T is the transformation function.

Logarithmic transform $s = c\log(1 + r)$, where $c = \frac{L-1}{\log(L)}$ expands the values of dark pixels while compressing higher-level values. It is primarily used to compress the dynamic range of an image with huge variations in pixel values, such as displaying the FT spectrum.

Power-law / gamma transform $s = cr^{\gamma}$, where $c = (L-1)^{1-\gamma}$ is more versatile than the log transform as by altering the gamma value, it can both expand dark regions (when $\gamma < 1$) or expand bright regions (when $\gamma > 1$). It is most commonly used for gamma correction to adjust an image's contrast to match the display specs.

Compare and contrast "Contrast Stretching" and "Thresholding".

Two techniques that both rely on piecewise-linear transformation functions to manipulate image contrast, but they serve different goals.

Contrast stretching: expands a narrow range of intensity levels in an image to span the full available dynamic range. It is used to correct low-contrast images by making the darks darker and brights brighter, while preserving the relative order of pixel intensities. It is represented as three segments, in which typically the one in middle has a slope coefficient > 1 resulting in a stretching of those intensity levels (contrast expansion), while the other two have slope coefficient < 1 resulting in an constrast compression.

Thresholding: extreme form of contrast stretching used for image segmentation. It maps all pixel intensities below a specified threshold value to a single dark value (like 0) and all intensities above the threshold to a single bright value (like L-1). Unlike stretching, thresholding irreversibly destroys the original intensity variations, resulting in a binary image.

What is the main limitation of Histogram Equalization, and why might we use Histogram Specification (Matching) instead?

The primary limitation of Histogram Equalization is that it is a fully automatic, global operation. It aggressively forces the image's histogram to be as flat as possible. While this generally increases contrast, it cannot be controlled. In images with uniform backgrounds or mostly dark/light areas, equalization can artificially enhance background noise or wash out important details.

Histogram Specification (Matching) solves this by giving the user control. Instead of forcing a flat histogram, it transforms the original image so that its histogram matches a specific, user-defined target histogram highlighting desired intensity ranges. This is useful for standardizing images from different sensors or preventing the over-enhancement caused by standard equalization.

Explain the concepts of histogram and its equalization. A) Define an image histogram. B) What is the goal of histogram equalization and why is it useful? C) Outline the procedure for deriving the equalization function, providing an example.

A) An image histogram is a graphical representation of the distribution of pixel intensities in an image. Specifically, it is a bar graph where the x-axis represents the possible intensity values (0 to L-1) and the y-axis represents the frequency (number of pixels) that assume each specific value. It provides a global description of image's appearance, showing if image is well contrasted, under/overexposed (histogram heavily concentrated on left/right side, respectively). It doesn't contain any spatial information about pixels location in the image.

B) Histogram Equalization is an image enhancement technique used to improve the global contrast of an image. It's useful when an image's pixels intensities are concentrated in a narrow range (e.g. under/overexposed images). The goal is to redistribute the pixel intensities so that they are as evenly spread as possible across the entire available spectrum, resulting in a flatter histogram and higher contrast.

C) That's achieved by computing PDF of original image so to find frequencies for all L intensity levels, then deriving CDF, then map original pixel values r_k to new equalized values. By replacing every pixel with a newly mapped value, we generate an image utilizing the full dynamic range.

Let's assume now to have a 3-bit image ($L = 8$), currently very dark (total 10 pixels):

Original intensity r_k	**Pixel count** n_k	**PDF** ($n_k/10$)	**CDF**	$(L-1)$**CDF**	s_k
0	2	0.2	0.2	1.4	1
1	4	0.4	0.6	4.2	4
2	3	0.3	0.9	6.3	6
3	1	0.1	1.0	7.0	7
4 to 7	0	0	1.0	7.0	7

Resulting in a brighter image, considering that the new equalized intensities s_k are greater than the original values r_k.

Describe the mathematical procedure for performing Histogram Specification.

Histogram Specification involves transforming an input image so its intensity distribution matches a specified target PDF. The procedure consists of 3 main steps: 1. Equalize the original image: compute the CDF of the original image to find the transformation $s = T(r)$. This maps the original pixels r to equalized values s. 2. Equalize the target histogram: compute CDF of the specified target histogram to find the transformation $v = G(z)$, $z =$ desired output pixel value. This maps the target pixels to equalized values v. 3. Apply the inverse transformation: Since both s, v are equalized (flat) distributions, let's assume $s \approx v$ and compute the inverse of the target transformation, $z = G^{-1}(s)$, and map the equalized pixels s from the first step to their final specified values z.

Lecture 6: Non-linear filters and frequency domain

Explain the principle of the median filter and its primary advantage over linear smoothing filters.

The median filter is a non-linear spatial filter that replaces the center pixel of a neighborhood with the median intensity value of the pixels within that window. Its primary advantage over linear filters (like the mean filter) is its superior ability to remove impulse noise (salt & pepper) without blurring the image excessively. While a mean filter averages the extreme noise values into the surrounding pixels, creating a blur, the median filter sorts the neighborhood values and picks the exact middle one, effectively discarding extreme outliers and preserving sharp edges (it doesn't compute any mean!).

How does the Bilateral Filter achieve edge-preserving smoothing, and how does it differ from a standard Gaussian filter?

A standard Gaussian filter smooths an image by computing a weighted average of neighboring pixels based solely on their spatial distance from the center. This reduces noise but blurs edges. The Bilateral Filter is a non-linear, edge-preserving technique that solves this by combining two Gaussian kernels: a spatial (domain) kernel and a range (intensity) kernel. The spatial kernel weights pixels based on physical proximity, while the range kernel weights them based on intensity similarity. Consequently, the bilateral filter only averages pixels that are both physically close and similar in brightness, ensuring flat regions are smoothed and preserving sharp intensity transitions / edges.

In the context of 2D FT, how do low and high frequency translate to the visual components of an image?

When an image is transformed into the frequency domain, its spatial intensity variations are represented as sinusoidal waves of varying frequencies. Low frequencies correspond to areas of the image where pixel intensities change slowly / are constant, i.e. smooth gradients, uniform background / illumination. They tipically contain the vast majority of the image's energy and appearance. High frequencies correspond to areas of rapid intensity transitions i.e. edges, fine details, random noise. Removing high frequencies results in blurring, while emphasizing them sharpens the image.

Compare ideal LPF and Butterworth LPF (BLPF) in the frequency domain. Why is BLPF generally preferred in practice?

Both filters are used in the frequency domain to attenuate high frequencies and smooth an image, but they differ in their cutoff characteristics. The ideal LPF has an infinitely sharp cutoff radius, passing all frequencies below the threshold perfectly and cutting off all the higher frequencies entirely. This truncation causes severe ringing artifacts (concentric ripples) in the resulting spatial image. The BLPF introduces a smooth transition between the passed and attenuated frequencies, controlled by the filter's order. Because it avoids

a sudden cutoff, a low-order BLPF significantly minimizes or completely eliminates the ringing effect, making it generally preferred for practical image processing and restoration. Note that as BLPF order grows to infinity, the BLPF approaches the ideal LPF, and ringing artifacts will return.

How are geometric spatial transformations mathematically represented and applied to an image?

Geometric transformations modify the spatial relationship between pixels (e.g. translating, rotating, scaling, shearing them). They are typically represented using affine transformations via transformation matrices and homogeneous coordinates. It allows us to combine multiple sequential operations, such as scaling an image and then rotating it, into a single composite transformation matrix. This matrix is then multiplied by the original pixel coordinates (x,y) to compute the new spatial locations.

Lecture 7: Filtering in frequency and geometrical transformations

Analyze the two steps required by a geometric trasform.

First is Coordinate transform which is a mathematical operation that works purely on geometrical points to determine their new spatial locations (represented as $(x', y') = T(x, y)$). The subsequent step is Image resampling that maps the actual pixel intensity values to the new discrete coordinate grid created by the transform.

How is a point represented in standard 2D coordinates vs homogeneous coordinates? And what is the structural advantage of using homogeneous coordinates vs standard cartesian ones when performing geometric translations?

In standard 2D coordinates a point is represented by two values (x, y). In homogeneous coordinates, an N-dimensional point is mapped into N+1 dimensions. For example, a 2D point (x, y) becomes a three-coordinate vector $(\tilde{x}, \tilde{y}, \tilde{w})$. You can always convert it back to standard coordinates by dividing by the scaling factor $\tilde{w}$, resulting in $x = \tilde{x}/\tilde{w}$ and $y = \tilde{y}/\tilde{w}$. In standard coordinates, an affine transformation requires separate linear operations and additions (e.g. multiplying by matrix A and adding translation vector b). By switching to homogeneous coordinates, multiple operations (including translation) can be easily combined into a single matrix multiplication.

What makes an affine transform different from a basic linear transform, and what specific properties does it preserve?

An Affine transform is a more generic planar transformation that consists of a linear transform followed by a translation. While it moves and scales points, an Affine transform specifically preserves point collinearity (points on a line stay on a line) and distance ratios along a line.

When analyzing an image for edges, what is the conceptual difference between using a derivative filter versus a true edge detection algorithm?

Derivative filters are a low-level operation; they simply highlight pixels where intensity changes rapidly. The resulting filtered image still lacks the actual concept of what an edge is. Edge detection algorithms, on the other hand, belong to mid-level processing because they mathematically embed the concept or "model" of an edge to actively detect and classify those structural features.

Lecture 8: Detecting edges and lines

How do first-order and second-order derivatives differ in their approach to edge detection?

Derivatives are used to detect rapid spatial changes in pixel intensity, which define edges. First-order derivatives (like the Sobel operator) compute the gradient of the image. An edge is identified by locating the local maxima (the highest peaks) in the gradient magnitude. The gradient also provides the direction of the edge. Second-order derivatives (like the Laplacian) compute the rate of change of the gradient. An edge is identified by locating zero-crossing: the exact point where the second derivative passes through zero. While second-order derivatives are better at identifying the exact center of an edge, they are exponentially more sensitive to high-frequency noise. Both derivatives amplify noise. For this reason, it's strictly necessary to smooth the image before applying edge detection. The most appropriate and used choice for this is a Gaussian Low-Pass Filter, which has a smooth derivative and it's also the foundation of Canny Edge Detector.

In the Canny algorithm pipeline, what distinct structural problems do Non-Maxima Suppression and Hysteresis Thresholding solve?

- **Non-Maxima Suppression** solves the problem of thick, blurred edge responses.

It ensures precise edge localization by thinning the gradient down to a single-pixel line. It achieves this by suppressing (setting to 0) any pixel that is not the local maximum along its quantized gradient direction.

- **Hysteresis Thresholding** solves the problem of fragmented edges and noise. By using two thresholds (T_H and T_L), it keeps strong edges, completely rejects isolated weak responses (noise), and most importantly, preserves weak edge pixels *only* if they are spatially connected to a strong edge, thus closing gaps.

How does the choice of the Gaussian kernel size (σ) in Step 1 dictate the final output of the Canny edge detector?

- **A small** σ applies minimal smoothing. This allows the detector to capture finer details and highly localized edges, though it makes the output more susceptible to keeping small noise components.
- **A large** σ applies aggressive smoothing. This blurs out fine textures and noise, restricting the detector to finding only coarse, large-scale, and highly prominent structural edges in the image.

Why is Canny considered a complete "edge detector algorithm" rather than just a simple derivative filter?

- A **simple derivative filter** (e.g., computing the gradient magnitude and applying a basic threshold) merely highlights areas of high intensity change. As seen in the slides, this typically results in thick, noisy, and broken boundary responses.
- The **Canny edge detector** embeds a complex, multi-step mathematical model of what an edge should be. It actively enforces three strict design criteria: a low error rate (via Gaussian smoothing), precise single-point localization (via non-maxima suppression), and continuous contour extraction (via hysteresis thresholding).

When building the parameter space for the Hough Transform, why is the normal representation ($x \cos\theta + y \sin\theta = \rho$) preferred over the standard Cartesian equation ($y = ax + b$)?

- **Cartesian Representation**: Maps an image point (x, y) to a line $b = -xa + y$ in the ab-parameter space. It fails to represent vertical lines because their slope (a) approaches infinity.

- **Normal Representation**: Solves the vertical line issue by mapping an image point to a bounded sinusoidal curve in the $\rho\theta$-parameter space. This allows the algorithm to gracefully detect lines at any angle.

In the Hough Transform algorithm, the parameter space (ρ, θ) must be quantized into accumulation cells. What is the engineering trade-off between using few cells versus many cells?

- **Few Cells (Coarse Quantization)**: Makes the algorithm robust to noise, allowing it to handle pixels that are not perfectly aligned along a mathematical line. The drawback is poor line localization.
- **Many Cells (Fine Quantization)**: Yields very accurate line localization, but requires the edge pixels in the image to be precisely aligned. If they are slightly scattered, they won't accumulate in the same cell, and the line might be missed.

How does the Generalized Hough Transform differ conceptually from the standard version, and what is its main computational drawback?

- **Standard Hough Transform**: Detects straight lines by finding intersections in a 2D parameter space (either ab or $\rho\theta$).
- **Generalized Hough Transform**: Extends the concept to find more complex shapes (like circles) using a general equation $g(\boldsymbol{v}, \boldsymbol{c}) = 0$, where $\boldsymbol{v}$ are coordinates and $\boldsymbol{c}$ are coefficients. For example, a circle uses $(x-c_1)^2+(y-c_2)^2 = c_3^2$. The main drawback is that complex shapes require a parameter space with **high dimensionality** (e.g., 3 dimensions for a circle), drastically increasing the computational and memory cost.

Why do we prefer the Hough Transform over a naive approach to find lines? (Computational Complexity)

- A **naive approach** would consider all pairs of edge points, evaluate the line passing through them, and count the points on that line. This yields a heavy computational complexity of $O(n^3)$ (considering $O(n^2)$ distinct pairs of points multiplied by a linear check to count the remaining points on the line).

- The **Hough Transform** avoids this combinatorics explosion by mapping each edge pixel independently into a parameter space and simply voting (accumulating) in a discrete grid, making line detection highly efficient.

Lecture 9: Morphological Operations and Thresholding

In morphological operations, how does Erosion differ from Dilation in terms of their physical effect on foreground shapes and component connectivity?

- **Erosion** shrinks or thins foreground objects. It is used to separate weakly connected components, as a pixel is kept only if the structuring element is *fully included* in the object set ($B_z \subseteq A$).
- **Dilation** expands or thickens foreground objects. It is used to merge close and unconnected components, as a pixel is kept if the structuring element has *at least one pixel overlapping* with the object set ($B_z \cap A \neq \emptyset$).

Structurally and mathematically, what is the exact difference between an Opening operation versus a Closing operation?

- **Opening** is defined mathematically as an Erosion followed by a Dilation: $A \circ B = (A \ominus B) \oplus B$. Structurally, it smooths the contour of an object, breaks narrow isthmuses, and eliminates thin protrusions.
- **Closing** is defined mathematically as a Dilation followed by an Erosion: $A \bullet B = (A \oplus B) \ominus B$. Structurally, it also smooths contours but does so by fusing narrow breaks and long thin gulfs, eliminating small holes, and filling gaps.

When dealing with imperfect binary images, how does Opening perform versus Closing in terms of noise removal and shape repair?

- **Opening** is best for "opening spaces" in the background: it removes small, bright (white) details or "salt" noise that are smaller than the structuring element, smoothing contours without reducing the main object's size.

- **Closing** is best for "closing gaps" in the foreground: it fills in small, dark (black) holes, narrow breaks, or "pepper" noise within the white objects, repairing the foreground without increasing the main object's overall size.

Since morphological operations are not cumulative, when is it appropriate to use [Opening then Closing] versus [Closing then Opening]?

- Use **Opening then Closing** when the background noise is severe but the main object is relatively sturdy (e.g., a noisy fingerprint). The Opening clears the background "salt" noise first so it doesn't get fused to the object, then the Closing patches cracks in the ridges.
- Use **Closing then Opening** when the main object is highly fragmented or full of holes (e.g., poorly printed dot-matrix text). You must Close first to fuse the broken pieces together; if you Opened first, the initial erosion step would completely erase the fragile object.

In the context of morphological descriptions, what is the conceptual difference between the image set versus the structuring element?

- The **image set** is the mathematical representation of the image itself. In the simplest case of a binary image, the image is defined as the set of coordinates containing the foreground pixels. Morphological operators act on this set by adding or removing pixels to/from it.
- The **structuring element** is the predefined "probe" or shape applied to the image. Its specific shape determines the direction(s) and extent to which pixels are added to or removed from the image set.

When segmenting an image, why might variable (local) thresholding be preferred over global thresholding?

- **Global thresholding** applies one single threshold value to the entire image. It works well only if the image has consistent lighting and a uniform background. It fails completely when the image suffers from uneven illumination, shadows, or gradients, because no single threshold can separate the foreground from the background everywhere.

- **Variable (or local) thresholding** divides the image into smaller sub-regions and calculates a distinct threshold for each specific area based on its local neighborhood. This allows the algorithm to dynamically adapt to local lighting changes, successfully segmenting objects even when the background intensity drifts across the image.

How does Otsu's method use inter-class variance versus intra-class variance to find the optimal threshold?

Otsu's method evaluates every possible threshold k, computes class probabilities and means, and picks the k that maximizes the inter-class variance σ_B^2. Since the global variance is fixed ($\sigma_G^2 = \sigma_{in}^2 + \sigma_B^2$), maximizing σ_B^2 is exactly equivalent to minimizing the intra-class variance σ_{in}^2, giving the optimal separation between the two classes.

In the context of Otsu's algorithm, what is the conceptual difference between the cumulative mean $m(k)$ and the mean intensity value $m_1(k)$ for Class 1?

- The **cumulative mean** $m(k) = \sum_{i=0}^{k} ip_i$ calculates the mean up to intensity level k, but it is normalized over the *whole* image.
- The **mean intensity value** for Class 1:

$$m_1(k) = \frac{1}{P_1(k)} \sum_{i=0}^{k} ip_i$$

  takes that cumulative mean and divides it by $P_1(k)$. This normalizes the value specifically to the number of pixels actually belonging to Class 1, giving the true average intensity of that segment.

How does image noise and object size affect the standard Otsu's method versus an approach that incorporates image smoothing?

- **Basic Otsu's method** struggles when dealing with noisy images or when target regions are very small (meaning they have a very limited impact on the histogram). Noise blurs the separation between histogram peaks.
- **Otsu's method with Smoothing** applies a spatial smoothing filter before calculating the histogram. This reduces noise, creates deeper valleys and clearer peaks in the histogram, and allows the algorithm to successfully segment smaller or noisier patterns.

Lecture 10: Segmentation by clustering

Compare region growing with thresholding-based segmentation, and outline the core steps of a basic region growing algorithm.

Standard segmentation criteria rely on either discontinuity (detecting boundaries/edges) or similarity (grouping alike pixels). While standard thresholding groups pixels globally based purely on intensity values without considering spatial relationships, region growing relies on local similarity and connectivity. It groups spatially contiguous pixels that share specific characteristics (intensity, color, texture) into larger regions. Algorithm Workflow:

1. **Initialization (Seeds):** Define the input image $f(x, y)$ and a seed array $S(x, y)$ containing 1s at seed locations and 0s elsewhere. If initial masks are obtained via rough heuristics (like global thresholding), each connected component is eroded to a single seed point. This removes the arbitrary shape of the initial mask, ensuring the final region is governed strictly by the growing predicate rather than the initialization method.

2. **Growing Predicate (Q):** Define a logical condition enforcing both *spatial connectivity* (e.g., an 8-connected neighborhood) and *similarity*. It evaluates to True (1) if the candidate pixel satisfies both criteria. For example, a common similarity check evaluates whether the pixel's intensity difference from the original seed is within a tolerance T:
$$Q(x_i, y_i) = |f(x_i, y_i) - f(x_s, y_s)| \leq T$$

3. **Growing Process:** Iteratively evaluate neighbors of the current region. If predicate Q is true, the pixel is appended to the region (creating a support image f_Q).

4. **Stopping Rule:** The region stops growing when no more neighboring pixels satisfy the predefined merging criteria.

5. **Output:** The final image f_G is generated, where each connected component is assigned a distinct region label.

Explain the core principle of Watershed segmentation.

The algorithm models a grayscale image (or its gradient magnitude) as a 3D topographic surface, where pixel intensity equals elevation.

- Flooding begins from local minima, forming expanding **catchment basins**.
- When water from distinct basins threatens to merge, a "dam" is mathematically constructed.

- The set of all dams constitutes the **watershed lines**, which serve as the final and fully connected segmentation boundaries.

What is the primary failure mode of Watershed segmentation, and what is the standard algorithmic enhancement used to resolve it?

Direct application on real-world images almost always causes severe oversegmentation. Image noise and minor local surface irregularities generate numerous spurious local minima, resulting in a highly fractured segmentation map comprised of tiny, useless regions. The standard solution is to abandon raw local minima and instead initiate flooding exclusively from a robust set of predefined **markers**.

- **Preprocessing:** The image is typically smoothed to suppress noise-induced variations.
- **Internal Markers:** Confirmed regions representing the actual objects of interest (e.g., prominent local minima surrounded by lighter pixels).
- **External Markers:** Regions firmly classified as the background.

By restricting the flooding seed points to these curated markers, the algorithm outputs a controlled number of meaningful, macroscopic regions.

Define the objective function of the K-means clustering algorithm and outline the standard iterative optimization approach (Lloyd's algorithm).

K-means aims to partition pixel feature vectors into k disjoint sets $C_1, \ldots, C_k$ with centroids μ_i, minimizing the intra-cluster variance (Sum of Squared Errors) made by approximating points with their cluster centers:

$$\min \left(\sum_{i=1}^{k} \sum_{\vec{x} \in C_i} d(\vec{x}, \mu_i) \right)$$

where $d(\cdot)$ is an appropriate distance metric. Because an exhaustive search is computationally unfeasible, a heuristic approach is required. Lloyd's Algorithm does iterative optimization:

1. **Initialization:** Select k initial centroids. Common heuristics include the *Forgy method* (randomly select k actual data points, ensuring centroids start within the dataset distribution) or *Random partition* (randomly assign all points to clusters then compute initial centroids).

2. **Assignment:** Associate each feature vector $\vec{x}$ to the closest centroid:

$$C_i = \{\vec{x} : i = \text{argmin}_j d(\vec{x}, \mu_j)\}$$

3. **Update:** Compute the new centroids as the center of mass (mean) of the associated points:

$$\mu_i = \frac{1}{|C_i|} \sum_{\vec{x} \in C_i} \vec{x}$$

4. **Termination:** Repeat assignment and update steps until a termination condition is achieved (centroids do not sensibly move, or a maximum step limit is reached).

5. **Output Visualization:** In the final segmented image, every pixel in a given cluster is typically reassigned the **mean intensity/color** of its respective cluster centroid.

Critically evaluate the structural assumptions of K-means segmentation, and analyze the trade-offs of including spatial coordinates in the feature vector versus using color/intensity alone.

Algorithmic Weaknesses & Assumptions:

- **Non-Optimality:** The heuristic guarantees fast convergence, but only to a local minimum. The output is strictly dependent on the initialization.
- **Rigid Parameters:** The exact number of clusters (k) must be known/provided in advance.
- **Geometric Bias (Spherical Symmetry):** The algorithm assumes data is evenly distributed around the centroid within a certain radius. It is mathematically incapable of accurately capturing real-world data structures that form elongated, non-convex, or highly irregular shapes in the feature space.

Feature Vector Composition Trade-offs:

- **Color/Intensity Only:** Segmented clusters will contain pixels that are **not necessarily spatially connected components** (e.g., similar objects on opposite sides of an image group together). It also fails critically on textured objects with high local variance (like cabbage), resulting in meaningless, highly fragmented clusters.
- **Color + Position (x, y):** Appending spatial coordinates enforces local compactness, vastly improving the separation of physically distinct objects. **Trade-off:** Large, homogeneous background regions will be artificially fractured into multiple distinct clusters simply because the spatial distance from a single centroid grows too large to minimize the objective function efficiently.

Explain the conceptual foundation of density-based clustering using Kernel Density Estimation and outline how it theoretically addresses K-means' limitations.

While K-means forms clusters based on the shortest distance to a calculated centroid (forcing spherical symmetry and requiring a predefined k), density-based clustering defines clusters as areas of high data point density separated by areas of lower density.

To find these areas, the algorithm constructs a continuous Probability Density Function (PDF) from the discrete pixel data using **Kernel Density Estimation (Parzen Window Technique)**:

1. **Kernel Placement:** A chosen kernel function (e.g., Uniform, Gaussian/Normal, Epanechnikov) of radius r is centered on every single sample point $\vec{x}_i$ in the dataset.
2. **Summation (Convolution):** The individual kernel contributions are summed across the feature space, blending the distributions to create a single, smooth global density estimate curve.
3. **Mode Seeking:** The algorithm then identifies the **modes** (the major peaks or local maxima) of this synthesized PDF. Each mode corresponds to a cluster center. All regions of the input space that "climb" toward the same peak are assigned to that specific cluster.

This approach theoretically solves major K-means flaws because it does not force arbitrary spherical shapes (clusters conform to the natural data topography), and the number of clusters is discovered organically based on the number of peaks found, rather than being guessed in advance.

Formally define the mathematical construction of a Probability Density Function using Kernel Density Estimation, and explain the primary computational drawback of this method.

Given N input samples $\vec{x}_i$ in an n-dimensional feature space, the estimated density function $f(\vec{x})$ evaluated at a generic point $\vec{x}$ is defined as the sum of translated kernels:

$$f(\vec{x}) = \frac{1}{Nr^n} \sum_{i=1}^{N} K(\vec{x} - \vec{x}_i) = \frac{c_k}{Nr^n} \sum_{i=1}^{N} k\left(\frac{||\vec{x} - \vec{x}_i||^2}{r^2}\right)$$

Where:

- $K(\cdot)$ is the specific kernel function (which must integrate to 1).
- $k(\cdot)$ is the 1-dimensional profile generating the kernel.
- r is the kernel radius (bandwidth), controlling the smoothness of the estimation.

- The factor r^n normalizes the equation by the number of dimensions to ensure the resulting PDF integrates to 1.

While theoretically robust, explicitly constructing and evaluating this global density function $f(\vec{x})$ everywhere across an n-dimensional feature space is **computationally highly complex and inefficient**. Calculating the contribution of every data point to every other coordinate in the space makes direct density function processing generally impractical for high-resolution image segmentation, necessitating alternative methods for finding peaks (like Mean Shift).